

[VN] Multi-tenant architecture nên thiết kế database như thế nào?

[EN] How should a database be designed for a multi-tenant architecture?

[VN] Trong multi-tenant architecture, có ba cách thiết kế database phổ biến. Cách thứ nhất là mỗi tenant có một database riêng, cách này cách ly dữ liệu tốt nhất và dễ bảo mật nhưng tốn tài nguyên và khó quản lý khi số tenant lớn. Cách thứ hai là mỗi tenant có một schema riêng trong cùng database, giúp cân bằng giữa cách ly dữ liệu và chi phí vận hành. Cách thứ ba là tất cả tenant dùng chung một schema và phân biệt bằng tenant_id trong mỗi bảng. Cách này tiết kiệm tài nguyên và dễ scale, nhưng cần đảm bảo filter tenant_id đúng trong mọi query và có index phù hợp để tối ưu hiệu năng. Tôi thường chọn shared schema với tenant_id cho hệ thống nhiều tenant nhỏ, và chọn database riêng khi yêu cầu bảo mật hoặc tuân thủ cao.

[EN] In multi-tenant architecture, there are three common database design approaches. The first is one database per tenant, which provides the strongest data isolation and security but consumes more resources and is harder to manage at large scale. The second is one schema per tenant in the same database, which balances isolation and operational cost. The third is a shared schema where all tenants share the same tables and are separated by a tenant_id column. This approach saves resources and scales well, but requires strict filtering by tenant_id in every query and proper indexing for performance. I usually choose shared schema with tenant_id for systems with many small tenants, and separate databases when strong security or compliance requirements are needed.

[VN] Làm sao viết test không phụ thuộc database thật?

[EN] How to write tests without relying on a real database?

[VN] Để viết test không phụ thuộc database thật, bạn nên tách business logic khỏi layer truy cập database và dùng mock hoặc fake repository để giả lập hành vi của database. Trong unit test, bạn chỉ test logic bằng cách inject dependency giả thay vì kết nối DB thật. Nếu cần test integration, có thể dùng in-memory database (như SQLite in-memory) hoặc tạo database riêng cho test và reset dữ liệu sau mỗi test. Cách này giúp test chạy nhanh, ổn định và không phụ thuộc môi trường bên ngoài.

[EN] To write tests without depending on a real database, you should separate business logic from the database access layer and use a mock or fake repository to simulate database behavior. In unit tests, you test only the logic by injecting fake dependencies instead of connecting to a real DB. For integration tests, you can use an in-memory database (such as SQLite in-memory) or create a dedicated test database and reset data after each test. This approach makes tests faster, more stable, and independent from external environments.

[VN] Index hoạt động thế nào trong PostgreSQL?

[EN] How do indexes work in PostgreSQL?

[VN] Index trong PostgreSQL là một cấu trúc dữ liệu giúp tăng tốc độ truy vấn bằng cách tránh phải quét toàn bộ bảng (full table scan). Mặc định, PostgreSQL thường sử dụng B-Tree index, hoạt động giống như một cây được sắp xếp, cho phép tìm kiếm, so sánh bằng (=), lớn hơn (>) hoặc nhỏ hơn (<) rất nhanh. Khi có index trên một cột, database sẽ tra cứu trong index trước để tìm vị trí dữ liệu, sau đó truy cập trực tiếp đến row tương ứng. Tuy nhiên, index cũng chiếm thêm dung lượng lưu trữ và làm chậm thao tác insert, update, delete vì phải cập nhật index. Vì vậy, cần tạo index đúng cột thường dùng trong WHERE, JOIN hoặc ORDER BY để đạt hiệu quả tốt nhất.

[EN] An index in PostgreSQL is a data structure that improves query performance by avoiding full table scans. By default, PostgreSQL commonly uses B-Tree indexes, which work like a sorted tree structure and allow fast search for operations such as equals (=), greater than (>), or less than (<). When a column has an index, the database looks up the value in the index first, then directly accesses the corresponding row in the table. However, indexes consume additional storage space and can slow down insert, update, and delete operations because the index must also be updated. Therefore, indexes should be created carefully on columns frequently used in WHERE, JOIN, or ORDER BY clauses for best performance.

[VN] Khi nào index làm chậm hệ thống?

[EN] When do indexes slow down the system?

[VN] Index có thể làm chậm hệ thống khi có quá nhiều index trên một bảng, đặc biệt với bảng có nhiều thao tác insert, update hoặc delete. Mỗi lần dữ liệu thay đổi, PostgreSQL phải cập nhật tất cả index liên quan, làm tăng chi phí ghi và giảm hiệu năng. Index cũng chiếm thêm bộ nhớ và dung lượng lưu trữ, có thể ảnh hưởng đến cache và I/O. Ngoài ra, nếu tạo index trên cột có độ phân biệt thấp (low cardinality) hoặc không thường xuyên dùng trong WHERE hoặc JOIN, query planner có thể không sử dụng index, dẫn đến lãng phí tài nguyên. Vì vậy, cần phân tích query thực tế và dùng EXPLAIN để tạo index hợp lý.

[EN] Indexes can slow down the system when there are too many indexes on a table, especially for tables with frequent insert, update, or delete operations. Every data change requires PostgreSQL to update all related indexes, which increases write cost and reduces performance. Indexes also consume additional memory and storage space, which can impact caching and I/O. Additionally, creating an index on a low-cardinality column or on columns rarely used in WHERE or JOIN clauses may not be beneficial, as the query planner may not use the index, leading to wasted resources. Therefore, it is important to analyze real queries and use EXPLAIN to create indexes properly.

[VN] Explain Analyze dùng để làm gì?

[EN] What is Explain Analyze used for?

[VN] Explain Analyze trong PostgreSQL được dùng để phân tích cách database thực thi một câu query và đo thời gian thực tế. Khi dùng EXPLAIN, hệ thống chỉ hiển thị execution plan, tức là kế hoạch mà query planner dự định sử dụng như index scan hay sequential scan. Khi dùng EXPLAIN ANALYZE, PostgreSQL sẽ thực thi query thật và hiển thị thêm thời gian thực tế, số row xử lý và số lần lặp. Nhờ đó, ta có thể so sánh estimated cost với actual time để tìm bottleneck, kiểm tra index có được sử dụng không và tối ưu query hiệu quả hơn.

[EN] EXPLAIN ANALYZE in PostgreSQL is used to analyze how the database executes a query and measure its real execution time. When using EXPLAIN, the system only shows the execution plan, meaning the plan that the query planner intends to use, such as index scan or sequential scan. When using EXPLAIN ANALYZE, PostgreSQL actually runs the query and provides additional details like real execution time, number of processed rows, and loop counts. This allows us to compare estimated cost with actual performance, identify bottlenecks, check whether indexes are used, and optimize queries more effectively.

[VN] Isolation level gồm những gì?

[EN] What are the isolation levels?

[VN] Isolation level là mức độ cô lập giữa các transaction trong database, quyết định cách các transaction nhìn thấy dữ liệu của nhau. Trong PostgreSQL có bốn mức chính: Read Uncommitted, Read Committed, Repeatable Read và Serializable. Read Uncommitted cho phép đọc dữ liệu chưa commit (dirty read), nhưng trong PostgreSQL nó được xử lý giống Read Committed. Read Committed chỉ cho phép đọc dữ liệu đã commit, nhưng vẫn có thể gặp non-repeatable read. Repeatable Read đảm bảo dữ liệu đã đọc trong transaction không thay đổi, nhưng vẫn có thể gặp phantom read tùy hệ quản trị. Serializable là mức cao nhất, đảm bảo kết quả giống như các transaction chạy tuần tự, nhưng có thể gây rollback nếu phát hiện xung đột. Mức isolation càng cao thì tính nhất quán càng mạnh nhưng hiệu năng có thể giảm.

[EN] Isolation level defines how transactions are isolated from each other in a database and how they see each other's data. In PostgreSQL, there are four main levels: Read Uncommitted, Read Committed, Repeatable Read, and Serializable. Read Uncommitted allows dirty reads, but in PostgreSQL it behaves the same as Read Committed. Read Committed only allows reading committed data, but non-repeatable reads can still occur. Repeatable Read ensures that data read within a transaction does not change, though

phantom reads may still happen depending on the system. Serializable is the highest level, ensuring transactions behave as if executed sequentially, but it may cause rollbacks if conflicts are detected. Higher isolation provides stronger consistency but may reduce performance.

[VN] Deadlock xảy ra khi nào?

[EN] When does a deadlock occur?

[VN] Deadlock xảy ra khi hai hoặc nhiều transaction giữ lock tài nguyên và đồng thời chờ nhau giải phóng lock, tạo thành vòng lặp chờ vô hạn. Ví dụ, transaction A lock row 1 và muốn lock row 2, trong khi transaction B đã lock row 2 và muốn lock row 1. Khi đó, cả hai sẽ chờ nhau và không thể tiếp tục. Trong PostgreSQL, hệ thống có cơ chế phát hiện deadlock và sẽ tự động hủy một transaction để giải quyết vòng lặp. Để tránh deadlock, nên lock tài nguyên theo cùng một thứ tự, giữ transaction ngắn gọn và tránh giữ lock lâu không cần thiết.

[EN] A deadlock happens when two or more transactions hold locks on resources and wait for each other to release them, creating an infinite waiting cycle. For example, transaction A locks row 1 and wants to lock row 2, while transaction B has locked row 2 and wants to lock row 1. In this case, both transactions wait for each other and cannot proceed. In PostgreSQL, the system can detect deadlocks and will automatically cancel one transaction to resolve the cycle. To prevent deadlocks, resources should be locked in a consistent order, transactions should be kept short, and locks should not be held longer than necessary.

[VN] Phân biệt optimistic vs pessimistic locking.

[EN] Differentiate optimistic vs pessimistic locking.

[VN] Optimistic locking là cơ chế giả định rằng xung đột dữ liệu hiếm khi xảy ra, nên không lock ngay từ đầu. Thay vào đó, khi update dữ liệu, hệ thống sẽ kiểm tra version hoặc timestamp để đảm bảo dữ liệu chưa bị thay đổi bởi transaction khác. Nếu version khác, transaction sẽ bị từ chối và phải retry. Cách này phù hợp với hệ thống có nhiều read và ít write, giúp tăng hiệu năng. Pessimistic locking thì ngược lại, hệ thống lock dữ liệu ngay khi đọc hoặc trước khi update để ngăn transaction khác truy cập. Cách này an toàn hơn khi xung đột thường xuyên xảy ra, nhưng có thể làm giảm concurrency và gây chờ đợi lâu hơn.

[EN] Optimistic locking assumes that data conflicts are rare, so it does not lock the data at the beginning. Instead, when updating, the system checks a version number or timestamp to ensure the data has not been changed by another transaction. If the version is different, the transaction fails and must retry. This approach works well in systems with many reads

and few writes, improving performance. Pessimistic locking, on the other hand, locks the data immediately when reading or before updating to prevent other transactions from accessing it. This approach is safer when conflicts happen frequently, but it can reduce concurrency and cause longer waiting times.

[VN] Transaction ACID là gì?

[EN] What is Transaction ACID?

[VN] Transaction ACID là bốn tính chất đảm bảo tính an toàn và nhất quán của dữ liệu trong database. ACID gồm: Atomicity, Consistency, Isolation và Durability. Atomicity nghĩa là transaction phải thực hiện toàn bộ hoặc không thực hiện gì cả, nếu có lỗi sẽ rollback. Consistency đảm bảo sau khi transaction hoàn thành, dữ liệu vẫn thỏa các ràng buộc và rule của hệ thống. Isolation đảm bảo các transaction chạy song song không ảnh hưởng sai lệch đến nhau theo mức isolation đã cấu hình. Durability đảm bảo khi transaction đã commit, dữ liệu sẽ không bị mất ngay cả khi hệ thống bị crash. Những tính chất này giúp hệ thống database đáng tin cậy và ổn định.

[EN] ACID is a set of four properties that ensure data safety and consistency in a database transaction. ACID stands for Atomicity, Consistency, Isolation, and Durability. Atomicity means a transaction must complete fully or not execute at all; if an error occurs, it is rolled back. Consistency ensures that after a transaction completes, the data still follows all rules and constraints of the system. Isolation ensures that concurrent transactions do not incorrectly affect each other based on the configured isolation level. Durability guarantees that once a transaction is committed, the data will not be lost even if the system crashes. These properties make the database system reliable and stable.

[VN] Partitioning trong PostgreSQL dùng khi nào?

[EN] When is partitioning used in PostgreSQL?

[VN] Partitioning trong PostgreSQL được dùng khi bảng có dữ liệu rất lớn và query bắt đầu chậm do phải scan nhiều row. Partitioning cho phép chia một bảng lớn thành nhiều partition nhỏ dựa trên một key như ngày tháng (range), giá trị cụ thể (list) hoặc hash. Khi query có điều kiện phù hợp với partition key, PostgreSQL chỉ scan các partition liên quan thay vì toàn bộ bảng, giúp tăng performance. Partitioning cũng hữu ích cho việc quản lý dữ liệu cũ như xóa hoặc archive theo từng partition. Tuy nhiên, cần thiết kế partition key hợp lý vì nếu chọn sai, hiệu năng có thể không cải thiện.

[EN] Partitioning in PostgreSQL is used when a table becomes very large and queries slow down due to scanning many rows. Partitioning splits a large table into smaller partitions based on a key such as date (range), specific values (list), or hash. When a query includes a condition on the partition key, PostgreSQL scans only the relevant partitions instead of the

entire table, which improves performance. Partitioning is also useful for managing old data, such as deleting or archiving by partition. However, the partition key must be designed carefully because a poor choice may not improve performance.

[VN] So sánh Redis vs Memcached.

[EN] Compare Redis vs Memcached.

[VN] Redis và Memcached đều là hệ thống in-memory dùng để cache dữ liệu nhằm tăng tốc ứng dụng. Memcached đơn giản, chỉ hỗ trợ key-value dạng string và tập trung vào caching thuần túy với hiệu năng cao và nhẹ. Redis mạnh hơn, ngoài key-value còn hỗ trợ nhiều kiểu dữ liệu như list, set, hash, sorted set và có thể dùng cho pub/sub, queue hoặc lưu session. Redis cũng hỗ trợ persistence (lưu dữ liệu xuống disk) và replication, trong khi Memcached chỉ lưu dữ liệu trong memory. Vì vậy, Memcached phù hợp cho caching đơn giản, còn Redis phù hợp khi cần tính năng nâng cao hoặc dùng như một data store nhẹ.

[EN] Redis and Memcached are both in-memory systems used for caching to improve application performance. Memcached is simple, supports only string key-value pairs, and focuses purely on high-performance caching with low overhead. Redis is more powerful; besides key-value, it supports multiple data types such as lists, sets, hashes, and sorted sets, and can be used for pub/sub, queues, or session storage. Redis also supports persistence to disk and replication, while Memcached stores data only in memory. Therefore, Memcached is suitable for simple caching, while Redis is better when advanced features or lightweight data storage are needed.

[VN] Redis Cluster và Redis Sentinel

[EN] Redis Cluster vs Redis Sentinel

[VN] Redis Sentinel và Redis Cluster đều giúp tăng độ sẵn sàng của Redis nhưng mục đích khác nhau. Redis Sentinel dùng để giám sát master và replica, tự động failover khi master bị lỗi, nhưng dữ liệu vẫn nằm trên một node master chính và không tự động chia shard. Redis Cluster vừa cung cấp high availability vừa tự động chia dữ liệu thành nhiều shard trên nhiều node, giúp scale ngang và tăng hiệu năng. Nói đơn giản, Sentinel tập trung vào failover và giám sát, còn Cluster tập trung vào vừa failover vừa phân tán dữ liệu để scale.

[EN] Redis Sentinel and Redis Cluster both improve Redis availability but serve different purposes. Redis Sentinel monitors master and replica nodes and performs automatic failover when the master fails, but the data still lives on a single main master and is not automatically sharded. Redis Cluster provides both high availability and automatic data sharding across multiple nodes, allowing horizontal scaling and better performance. In simple terms, Sentinel focuses on monitoring and failover, while Cluster focuses on both failover and data distribution for scaling.

[VN] Cache invalidation strategies?

[EN] Cache invalidation strategies?

[VN] Cache invalidation là cách đảm bảo dữ liệu trong cache không bị cũ hoặc sai so với database. Có một số chiến lược phổ biến. Thứ nhất là TTL (time-to-live), cache sẽ tự hết hạn sau một khoảng thời gian nhất định. Thứ hai là write-through hoặc write-back, khi dữ liệu được cập nhật thì cache cũng được cập nhật cùng lúc. Thứ ba là cache-aside, application sẽ xóa hoặc cập nhật cache khi có thay đổi trong database. Ngoài ra có thể dùng event-driven invalidation, khi một service publish event thay đổi dữ liệu thì các service khác sẽ xóa cache liên quan. Mỗi cách có trade-off giữa độ đơn giản, độ chính xác và hiệu năng.

[EN] Cache invalidation is the process of ensuring that cached data does not become stale or inconsistent with the database. There are several common strategies. First is TTL (time-to-live), where the cache automatically expires after a fixed time. Second is write-through or write-back, where the cache is updated together with the database write. Third is cache-aside, where the application deletes or updates the cache when the database changes. Another approach is event-driven invalidation, where a service publishes an event when data changes, and other services clear related cache entries. Each strategy has trade-offs between simplicity, consistency, and performance.

[VN] N+1 query problem là gì?

[EN] What is the N+1 query problem?

[VN] N+1 query problem là vấn đề xảy ra khi application thực hiện 1 query để lấy danh sách dữ liệu (N record), sau đó với mỗi record lại thực hiện thêm 1 query khác để lấy dữ liệu liên quan. Kết quả là tổng cộng có $1 + N$ query, gây tốn nhiều thời gian và làm chậm hệ thống, đặc biệt khi N lớn. Vấn đề này thường gặp khi dùng ORM với quan hệ one-to-many hoặc many-to-one. Cách giải quyết là dùng join, eager loading, hoặc batch query để lấy dữ liệu liên quan trong một hoặc ít query hơn.

[EN] The N+1 query problem happens when an application runs one query to get a list of records (N records), and then runs one additional query for each record to fetch related data. As a result, it executes $1 + N$ queries, which increases latency and slows down the system, especially when N is large. This problem is common when using ORM with one-to-many or many-to-one relationships. The solution is to use joins, eager loading, or batch queries to fetch related data in one or fewer queries.

[VN] Thiết kế schema thế nào để tránh N+1 query?

[EN] How to design schema to avoid N+1 query?

[VN] Để tránh N+1 query, khi thiết kế schema nên xác định rõ các quan hệ giữa bảng (foreign key) và đảm bảo có index trên các cột liên kết để JOIN nhanh. Thay vì thiết kế để phải truy vấn từng bản ghi riêng lẻ, nên tối ưu cho việc lấy dữ liệu theo tập (set-based), ví dụ dùng JOIN hoặc eager loading trong ORM để lấy dữ liệu liên quan trong một query. Ngoài ra, có thể denormalize một số dữ liệu thường dùng chung để giảm số lần truy vấn. Tóm lại, thiết kế schema và cách truy vấn phải hướng đến việc lấy dữ liệu theo nhóm thay vì từng dòng một để tránh N+1.

[EN] To avoid N+1 queries, when designing the schema you should clearly define table relationships (using foreign keys) and create indexes on join columns for fast JOIN operations. Instead of designing in a way that requires querying each record separately, optimize for set-based data access, for example by using JOINS or eager loading in an ORM to fetch related data in a single query. You may also denormalize some frequently accessed data to reduce extra queries. In short, both schema design and query strategy should focus on fetching data in batches rather than row by row to prevent the N+1 problem.

[VN] Khi nào dùng NoSQL như DynamoDB?

[EN] When to use NoSQL like DynamoDB?

[VN] Nên dùng NoSQL như DynamoDB khi hệ thống cần scale rất lớn, latency thấp và workload có nhiều read/write với cấu trúc dữ liệu linh hoạt. DynamoDB phù hợp với mô hình key-value hoặc document, không cần join phức tạp như relational database. Nó thích hợp cho hệ thống microservices, session store, caching layer, hoặc các ứng dụng có traffic cao và cần auto scaling. Ngoài ra, DynamoDB phù hợp khi cần high availability và quản lý hạ tầng tối thiểu vì là managed service. Tuy nhiên, nếu hệ thống cần transaction phức tạp hoặc query quan hệ nhiều bảng, database quan hệ có thể phù hợp hơn.

[EN] You should use NoSQL like DynamoDB when the system needs massive scalability, low latency, and handles heavy read/write workloads with flexible data structures. DynamoDB works well with key-value or document models and does not support complex joins like relational databases. It is suitable for microservices, session storage, caching layers, or high-traffic applications that require auto scaling. It is also a good choice when you need high availability and minimal infrastructure management because it is a managed service. However, if the system requires complex transactions or relational queries across many tables, a relational database may be a better choice.

[VN] Cách thiết kế schema cho multi-tenant.

[EN] How to design schema for multi-tenant.

[VN] Thiết kế schema cho multi-tenant có ba cách phổ biến. Cách thứ nhất là shared database, shared schema, tất cả tenant dùng chung bảng và phân biệt bằng tenant_id; cách này dễ scale và tiết kiệm chi phí nhưng cần đảm bảo bảo mật và index tốt. Cách thứ hai là shared database, separate schema, mỗi tenant có một schema riêng trong cùng database; cách này cô lập dữ liệu tốt hơn nhưng quản lý phức tạp hơn. Cách thứ ba là separate database cho mỗi tenant; cách này cô lập cao nhất và phù hợp với khách hàng lớn, nhưng tốn tài nguyên và chi phí hơn. Việc chọn cách nào phụ thuộc vào số lượng tenant, yêu cầu bảo mật, khả năng scale và chi phí vận hành.

[EN] There are three common ways to design a schema for multi-tenant systems. The first is shared database, shared schema, where all tenants use the same tables and are separated by a tenant_id; this approach is cost-effective and scalable but requires strong security and proper indexing. The second is shared database, separate schema, where each tenant has its own schema in the same database; this provides better data isolation but is more complex to manage. The third is separate database per tenant; this gives the highest isolation and is suitable for large customers, but it requires more resources and higher cost. The choice depends on the number of tenants, security requirements, scalability needs, and operational cost.

[VN] Replication trong PostgreSQL hoạt động thế nào?

[EN] How does replication work in PostgreSQL?

[VN] Replication trong PostgreSQL là cơ chế sao chép dữ liệu từ primary server sang một hoặc nhiều replica server để tăng high availability và khả năng đọc. Cách phổ biến nhất là streaming replication, trong đó primary ghi thay đổi vào WAL (Write-Ahead Log), sau đó replica sẽ nhận và apply các WAL này để giữ dữ liệu đồng bộ. Replication có thể là synchronous, khi primary chờ replica xác nhận trước khi commit, hoặc asynchronous, khi primary commit ngay và replica cập nhật sau. Replication giúp tăng khả năng chịu lỗi và scale read, nhưng replica thường chỉ đọc được, không dùng để ghi.

[EN] Replication in PostgreSQL is a mechanism that copies data from a primary server to one or more replica servers to improve high availability and read scalability. The most common method is streaming replication, where the primary writes changes to the WAL (Write-Ahead Log), and replicas receive and apply these WAL records to stay in sync. Replication can be synchronous, where the primary waits for replica confirmation before commit, or asynchronous, where the primary commits immediately and replicas update later. Replication increases fault tolerance and read scaling, but replicas are usually read-only and not used for writes.

[VN] Connection pool là gì? Tại sao quan trọng?

[EN] What is a connection pool? Why is it important?

[VN] Connection pool là cơ chế quản lý và tái sử dụng các kết nối đến database thay vì tạo và đóng kết nối mới cho mỗi request. Khi application cần truy cập database, nó sẽ lấy một connection có sẵn trong pool và trả lại sau khi dùng xong. Việc tạo connection mới rất tốn tài nguyên và thời gian, vì vậy connection pool giúp giảm latency, tăng hiệu năng và kiểm soát số lượng connection tối đa đến database. Điều này rất quan trọng trong hệ thống có nhiều request đồng thời, vì nếu tạo quá nhiều connection có thể làm quá tải database và gây lỗi.

[EN] Connection pool is a mechanism that manages and reuses database connections instead of creating and closing a new connection for every request. When the application needs to access the database, it takes an available connection from the pool and returns it after use. Creating new connections is expensive in terms of time and resources, so connection pooling reduces latency, improves performance, and controls the maximum number of connections to the database. This is very important in high-concurrency systems, because too many connections can overload the database and cause failures.

[VN] Seq Scan vs Index Scan khác nhau thế nào và khi nào Seq Scan lại nhanh hơn?

[EN] How do Seq Scan vs Index Scan differ and when is Seq Scan faster?

[VN] Seq Scan (Sequential Scan) là cách PostgreSQL đọc toàn bộ bảng từ đầu đến cuối rồi lọc dữ liệu theo điều kiện. Index Scan là cách dùng index để tìm nhanh những dòng phù hợp rồi mới truy cập vào bảng để lấy dữ liệu chi tiết. Index Scan thường nhanh hơn khi truy vấn chỉ lấy một phần nhỏ dữ liệu (ví dụ vài phần trăm số dòng). Tuy nhiên, Seq Scan có thể nhanh hơn khi truy vấn cần đọc phần lớn hoặc toàn bộ bảng, khi bảng rất nhỏ, hoặc khi điều kiện không chọn lọc cao. Lý do là đọc tuần tự giúp tận dụng I/O liên tục và tránh chi phí truy cập index nhiều lần.

[EN] Seq Scan (Sequential Scan) is when PostgreSQL reads the whole table from start to end and then filters the rows. Index Scan uses an index to quickly find matching rows and then fetches the actual data from the table. Index Scan is usually faster when the query returns a small percentage of rows. However, Seq Scan can be faster when the query needs most or all rows, when the table is small, or when the condition is not very selective. This is because sequential reading is efficient for disk I/O and avoids the extra cost of many index lookups.

[VN] Index B-tree, Hash, GIN, GiST khác nhau thế nào và dùng khi nào?

[EN] How do B-tree, Hash, GIN, and GiST indexes differ and when are they used?

[VN] Trong PostgreSQL, B-tree là loại index mặc định, phù hợp cho so sánh bằng, lớn hơn, nhỏ hơn và dùng tốt cho hầu hết query thông thường như tìm theo id hoặc range.

Hash index tối ưu cho so sánh bằng (=) nhưng ít dùng hơn vì B-tree cũng hỗ trợ tốt và linh hoạt hơn. GIN phù hợp cho dữ liệu phức tạp như array, JSONB hoặc full-text search vì nó index từng phần tử bên trong cấu trúc dữ liệu. GiST linh hoạt hơn, thường dùng cho dữ liệu không gian (geospatial) hoặc các kiểu dữ liệu cần so sánh đặc biệt. Tóm lại, B-tree cho nhu cầu phổ biến, Hash cho so sánh bằng đơn giản, GIN cho dữ liệu nhiều giá trị bên trong, và GiST cho dữ liệu đặc biệt như không gian.

[EN] In PostgreSQL, B-tree is the default index type and is suitable for equality and range comparisons, working well for most common queries such as searching by id or ranges. Hash index is optimized for equality (=) comparisons but is less commonly used because B-tree already handles this well and is more flexible. GIN is suitable for complex data types like arrays, JSONB, or full-text search because it indexes individual elements inside the data structure. GiST is more flexible and is often used for spatial (geospatial) data or special comparison needs. In short, B-tree is for general use, Hash for simple equality, GIN for multi-value or document-like data, and GiST for special data such as spatial types.

[VN] Index có giúp cho LIKE hoặc ILIKE không?

[EN] Do indexes help with LIKE or ILIKE?

[VN] Index có thể giúp cho LIKE hoặc ILIKE nhưng tùy cách dùng. Với B-tree index, nó chỉ hiệu quả khi pattern có dạng “prefix%” (ví dụ name LIKE 'abc%') vì database có thể dùng index để tìm theo thứ tự. Nếu pattern bắt đầu bằng ký tự đại diện như “%abc” thì B-tree không dùng được. Với ILIKE (không phân biệt hoa thường), thường cần tạo index trên lower(column) hoặc dùng extension như pg_trgm để hỗ trợ tìm kiếm gần đúng và pattern phức tạp. Tóm lại, index giúp cho LIKE khi có prefix rõ ràng, còn pattern linh hoạt hơn cần kỹ thuật hoặc index đặc biệt.

[EN] An index can help with LIKE or ILIKE, but it depends on how it is used. With a B-tree index, it works efficiently only when the pattern is a prefix search like “prefix%” (for example, name LIKE 'abc%') because the database can use the sorted order of the index. If the pattern starts with a wildcard like “%abc”, a B-tree index cannot be used. For ILIKE (case-insensitive search), you usually need to create an index on lower(column) or use extensions like pg_trgm to support more flexible or fuzzy patterns. In short, an index helps LIKE when there is a clear prefix, while more flexible patterns require special techniques or indexes.

[VN] Khi nào PostgreSQL không sử dụng index dù đã tạo?

[EN] When does PostgreSQL not use an index even if it has been created?

[VN] PostgreSQL có thể không sử dụng index dù đã tạo khi planner ước tính rằng quét toàn bảng (sequential scan) rẻ hơn, ví dụ khi bảng nhỏ hoặc khi query trả về phần lớn dữ

liệu. Index cũng không được dùng nếu điều kiện không phù hợp với loại index, như dùng hàm trên cột nhưng không có functional index, hoặc dùng LIKE với “%abc”. Ngoài ra, thống kê (statistics) cũ hoặc không chính xác cũng làm planner chọn kế hoạch sai. Tóm lại, PostgreSQL dựa trên cost estimation, nên nếu dùng index không giúp giảm chi phí thì nó sẽ bỏ qua.

[EN] PostgreSQL may not use an index even if it exists when the query planner estimates that a sequential scan is cheaper, for example when the table is small or the query returns a large portion of rows. An index is also not used if the condition does not match the index type, such as applying a function to a column without a functional index, or using LIKE with “%abc”. Outdated or inaccurate statistics can also cause the planner to choose a different plan. In short, PostgreSQL uses cost estimation, so if the index does not reduce the cost, it will ignore it.

[VN] Batch insert vs single insert khác nhau thế nào?

[EN] How do batch insert vs single insert differ?

[VN] Batch insert là chèn nhiều bản ghi trong một câu lệnh hoặc một transaction, còn single insert là chèn từng bản ghi riêng lẻ. Batch insert nhanh hơn vì giảm số lần kết nối, giảm round-trip giữa ứng dụng và database, và giảm overhead của transaction commit. Single insert đơn giản hơn nhưng nếu chèn số lượng lớn dữ liệu thì sẽ chậm hơn và tốn tài nguyên hơn. Tóm lại, khi cần ghi nhiều dữ liệu cùng lúc, nên dùng batch insert để tối ưu hiệu năng.

[EN] Batch insert means inserting multiple records in one statement or one transaction, while single insert inserts one record at a time. Batch insert is faster because it reduces the number of connections, network round-trips between the application and the database, and transaction commit overhead. Single insert is simpler but becomes slower and more resource-intensive when inserting a large amount of data. In short, when you need to write many records at once, batch insert is better for performance.

[VN] Làm sao tối ưu insert/update hàng loạt (bulk operation)?

[EN] How to optimize bulk insert/update operations?

[VN] Để tối ưu insert hoặc update hàng loạt trong database, bạn nên dùng batch operation thay vì xử lý từng dòng một, ví dụ dùng bulk insert, COPY (trong PostgreSQL) hoặc executemany trong driver. Nên gom nhiều thao tác vào một transaction để giảm chi phí commit. Với update, có thể dùng câu lệnh UPDATE với JOIN hoặc CASE WHEN thay vì loop trong application. Ngoài ra, tạm thời tắt index không cần thiết hoặc giảm constraint khi import dữ liệu lớn cũng giúp tăng tốc (nếu phù hợp). Tóm lại, mục tiêu là giảm round-trip, giảm commit nhiều lần và tận dụng khả năng xử lý tập trung của database.

[EN] To optimize bulk insert or update operations, you should use batch operations instead of processing one row at a time, such as bulk insert, COPY (in PostgreSQL), or executemany in the driver. Group many operations into a single transaction to reduce commit overhead. For updates, use a single UPDATE statement with JOIN or CASE WHEN instead of looping in the application. You can also temporarily disable unnecessary indexes or reduce constraints when importing large datasets (if appropriate). In short, the goal is to reduce network round-trips, minimize multiple commits, and leverage the database's set-based processing power.

[VN] Khi nào transaction quá dài gây vấn đề?

[EN] When does a transaction being too long cause problems?

[VN] Transaction quá dài gây vấn đề khi nó giữ lock trên bảng hoặc dòng dữ liệu quá lâu, làm các transaction khác bị chờ hoặc gây deadlock. Trong PostgreSQL, transaction dài cũng giữ lại các phiên bản dữ liệu cũ (MVCC), làm tăng kích thước bảng, chậm VACUUM và giảm hiệu năng hệ thống. Ngoài ra, nếu transaction kéo dài qua nhiều thao tác I/O hoặc xử lý logic phức tạp, nguy cơ lỗi và rollback sẽ tồn kém hơn. Tóm lại, transaction nên ngắn gọn, chỉ bao gồm các thao tác cần tính nhất quán và commit sớm nhất có thể.

[EN] A transaction becomes problematic when it holds locks on tables or rows for too long, causing other transactions to wait or even leading to deadlocks. In PostgreSQL, long transactions also keep old row versions (due to MVCC), which increases table size, slows down VACUUM, and reduces system performance. If a transaction spans many I/O operations or complex logic, the risk of failure and the cost of rollback also increase. In short, transactions should be short, include only necessary consistent operations, and commit as early as possible.

[VN] Lock và blocking ảnh hưởng performance ra sao?

[EN] How do locks and blocking affect performance?

[VN] Lock và blocking ảnh hưởng performance khi một transaction giữ lock trên dữ liệu và transaction khác phải chờ, làm tăng thời gian phản hồi và giảm throughput hệ thống. Khi nhiều request cùng chờ một lock, CPU có thể nhàn rỗi nhưng ứng dụng vẫn chậm vì bị “kẹt” ở database. Blocking kéo dài còn có thể gây timeout, deadlock và làm user thấy hệ thống treo. Tóm lại, lock là cần thiết để đảm bảo tính nhất quán, nhưng nếu không thiết kế tốt thì blocking sẽ làm giảm hiệu năng và khả năng mở rộng.

[EN] Locks and blocking affect performance when one transaction holds a lock on data and other transactions must wait, increasing response time and reducing system throughput. When many requests wait for the same lock, the CPU may be idle but the

application still feels slow because it is stuck at the database level. Long blocking can also cause timeouts, deadlocks, and make the system appear frozen to users. In short, locks are necessary for consistency, but poor design can lead to blocking that reduces performance and scalability.

[VN] Cách kiểm tra query đang bị lock?

[EN] How to check which query is being locked?

[VN] Để kiểm tra query đang bị lock trong PostgreSQL, bạn có thể xem các view hệ thống như `pg_stat_activity` để biết query nào đang chạy và trạng thái của nó, và `pg_locks` để xem các lock đang được giữ hoặc đang chờ. Bằng cách join `pg_stat_activity` với `pg_locks`, bạn có thể xác định session nào đang blocking và session nào đang bị chờ. Ngoài ra, có thể dùng hàm `pg_blocking_pids()` để tìm process đang chặn một process khác. Tóm lại, dựa vào các view hệ thống và phân tích quan hệ giữa các session để tìm nguyên nhân lock.

[EN] To check if a query is being locked in PostgreSQL, you can use system views such as `pg_stat_activity` to see running queries and their states, and `pg_locks` to view current locks that are held or waiting. By joining `pg_stat_activity` with `pg_locks`, you can identify which session is blocking and which one is waiting. You can also use the function `pg_blocking_pids()` to find the process that is blocking another process. In short, you analyze system views and the relationship between sessions to find the source of the lock.

[VN] Khi nào nên dùng composite index và thứ tự cột trong index quan trọng ra sao?

[EN] When should composite indexes be used and how important is the column order in an index?

[VN] Composite index (index nhiều cột) nên dùng khi query thường lọc hoặc sắp xếp theo nhiều cột cùng lúc, ví dụ `WHERE col1 = ? AND col2 = ?`. Nó giúp database tìm dữ liệu nhanh hơn so với nhiều index đơn lẻ. Thứ tự cột trong composite index rất quan trọng vì database chỉ tận dụng tốt khi điều kiện query phù hợp với thứ tự từ trái sang phải (leftmost prefix). Cột có độ chọn lọc cao hoặc thường xuất hiện trong điều kiện lọc nên đặt trước. Tóm lại, dùng composite index khi các cột hay đi cùng nhau trong query, và sắp xếp thứ tự dựa trên cách truy vấn thực tế.

[EN] A composite index (multi-column index) should be used when queries often filter or sort by multiple columns together, for example `WHERE col1 = ? AND col2 = ?`. It helps the database find data faster compared to using separate single-column indexes. The order of columns in a composite index is very important because the database can effectively use it only when the query matches the leftmost prefix order. Columns with high selectivity or those frequently used in filters should come first. In short, use a composite index when

columns are commonly queried together, and choose the column order based on real query patterns.

[VN] Covering index (index only scan) là gì?

[EN] What is a covering index (index only scan)?

[VN] Covering index (index only scan) là khi database có thể trả kết quả chỉ bằng cách đọc dữ liệu từ index mà không cần truy cập vào bảng chính (table). Điều này xảy ra khi tất cả các cột cần trong câu query (SELECT, WHERE, ORDER BY) đều nằm trong index. Khi đó, database tiết kiệm I/O, giảm số lần đọc disk và tăng hiệu năng. Tóm lại, covering index giúp query nhanh hơn vì không cần “quay về” table để lấy thêm dữ liệu.

[EN] A covering index (index only scan) happens when the database can return the result using only the index without accessing the main table. This is possible when all columns required by the query (SELECT, WHERE, ORDER BY) are included in the index. In this case, the database reduces disk I/O and improves performance. In short, a covering index makes queries faster because it does not need to go back to the table to fetch extra data.

[VN] Bitmap Index Scan dùng khi nào?

[EN] When is Bitmap Index Scan used?

[VN] Bitmap Index Scan được dùng khi query trả về nhiều dòng nhưng không quá nhiều đến mức phải quét toàn bảng, hoặc khi kết hợp nhiều index trong một điều kiện phức tạp (ví dụ nhiều điều kiện AND/OR). PostgreSQL sẽ đọc nhiều index, tạo bitmap để đánh dấu các vị trí dòng phù hợp, sau đó mới truy cập table theo từng block thay vì từng dòng riêng lẻ. Cách này giúp giảm số lần truy cập ngẫu nhiên vào disk và tối ưu I/O. Tóm lại, Bitmap Index Scan phù hợp khi cần lấy số lượng dòng trung bình hoặc khi kết hợp nhiều index cùng lúc.

[EN] Bitmap Index Scan is used when a query returns many rows but not so many that a full table scan is better, or when combining multiple indexes in complex conditions (such as multiple AND/OR clauses). PostgreSQL reads several indexes, builds a bitmap to mark matching row locations, and then accesses the table by blocks instead of row by row. This reduces random disk access and optimizes I/O. In short, Bitmap Index Scan is suitable for medium result sets or when combining multiple indexes together.

[VN] Subquery vs JOIN khác nhau về hiệu năng ra sao?

[EN] How do Subquery vs JOIN differ in terms of performance?

[VN] Về hiệu năng, subquery và JOIN có thể cho kết quả tương đương vì PostgreSQL thường rewrite subquery thành JOIN trong quá trình tối ưu. Tuy nhiên, trong một số trường hợp như correlated subquery (subquery phụ thuộc từng dòng), nó có thể bị chạy

nhiều lần và gây chậm hơn so với JOIN. JOIN thường rõ ràng và dễ tối ưu hơn, đặc biệt khi có index phù hợp. Tóm lại, không phải lúc nào JOIN cũng nhanh hơn, nhưng JOIN thường ổn định và dễ kiểm soát hiệu năng hơn so với subquery phức tạp.

[EN] In terms of performance, subqueries and JOINS can produce similar results because PostgreSQL often rewrites subqueries into JOINS during optimization. However, in cases like correlated subqueries (which depend on each row), they may be executed multiple times and become slower than a JOIN. JOINS are usually clearer and easier to optimize, especially when proper indexes exist. In short, JOIN is not always faster, but it is generally more stable and easier to control in terms of performance compared to complex subqueries.

[VN] Làm sao tối ưu truy vấn có ORDER BY và LIMIT?

[EN] How to optimize queries with ORDER BY and LIMIT?

[VN] Để tối ưu truy vấn có ORDER BY và LIMIT, nên tạo index phù hợp với cột dùng để sắp xếp, đặc biệt là index cùng thứ tự (ASC/DESC) với query. Nếu điều kiện WHERE và ORDER BY thường đi cùng nhau, có thể dùng composite index để database vừa lọc vừa sắp xếp trên index mà không cần sort thêm. Điều này giúp tránh full sort và giảm I/O. Ngoài ra, LIMIT nhỏ sẽ có lợi nếu index được dùng vì database có thể dừng sớm khi đủ số dòng cần thiết. Tóm lại, index đúng cột và đúng thứ tự là yếu tố quan trọng để tối ưu ORDER BY và LIMIT.

[EN] To optimize a query with ORDER BY and LIMIT, you should create an index on the column used for sorting, especially with the same order (ASC/DESC) as in the query. If WHERE and ORDER BY are often used together, a composite index can help the database filter and sort directly from the index without an extra sort step. This avoids full sorting and reduces I/O. A small LIMIT is also beneficial when using an index because the database can stop early after fetching enough rows. In short, the correct index with the right column order is key to optimizing ORDER BY and LIMIT.

[VN] Làm sao tối ưu truy vấn có GROUP BY và aggregation lớn?

[EN] How to optimize queries with GROUP BY and large aggregations?

[VN] Để tối ưu truy vấn có GROUP BY và aggregation lớn, nên tạo index trên các cột dùng trong GROUP BY hoặc trong điều kiện WHERE để giảm số dòng cần xử lý trước khi gom nhóm. Cố gắng lọc dữ liệu sớm (WHERE trước GROUP BY) để giảm khối lượng tính toán. Với dữ liệu rất lớn, có thể dùng partition table để chia nhỏ dữ liệu, hoặc tạo materialized view để lưu sẵn kết quả tổng hợp nếu query chạy thường xuyên. Ngoài ra, đảm bảo statistics được cập nhật để planner chọn execution plan tốt. Tóm lại, giảm số dòng đầu vào và tận dụng index, partition hoặc pre-aggregation là cách tối ưu chính.

[EN] To optimize queries with GROUP BY and large aggregations, create indexes on columns used in GROUP BY or WHERE clauses to reduce the number of rows processed before grouping. Filter data as early as possible (apply WHERE before GROUP BY) to minimize computation. For very large datasets, consider table partitioning to split data into smaller parts, or use materialized views to store pre-aggregated results if the query runs frequently. Also ensure statistics are up to date so the planner can choose a good execution plan. In short, reduce input rows and leverage indexes, partitioning, or pre-aggregation for better performance.

[VN] Window function ảnh hưởng hiệu năng thế nào?

[EN] How do window functions affect performance?

[VN] Window function có thể ảnh hưởng hiệu năng vì nó thường yêu cầu sắp xếp dữ liệu theo PARTITION BY và ORDER BY trước khi tính toán, điều này tốn CPU và bộ nhớ, đặc biệt với tập dữ liệu lớn. Nếu không có index phù hợp cho cột dùng để sắp xếp, database phải thực hiện full sort, làm tăng thời gian xử lý. Ngoài ra, window function không giảm số dòng như GROUP BY mà vẫn giữ nguyên số dòng ban đầu, nên lượng dữ liệu xử lý có thể rất lớn. Tóm lại, window function rất mạnh và tiện lợi, nhưng cần chú ý index và kích thước dữ liệu để tránh giảm hiệu năng.

[EN] Window functions can impact performance because they often require sorting data based on PARTITION BY and ORDER BY before calculation, which consumes CPU and memory, especially with large datasets. If there is no proper index on the sorting columns, the database must perform a full sort, increasing processing time. Unlike GROUP BY, window functions do not reduce the number of rows and keep the original row count, so the amount of processed data can be large. In short, window functions are powerful and useful, but you must consider indexes and data size to avoid performance issues.

[VN] Bạn hiểu thế nào về query planner và cost-based optimizer trong PostgreSQL?

[EN] How do you understand the query planner and cost-based optimizer in PostgreSQL?

[VN] Query planner trong PostgreSQL là thành phần quyết định cách thực thi một câu lệnh SQL, ví dụ dùng index scan hay sequential scan, join theo cách nào. Cost-based optimizer là cơ chế ước tính “chi phí” của nhiều execution plan khác nhau dựa trên thống kê dữ liệu (statistics), số dòng dự đoán, chi phí I/O và CPU, rồi chọn plan có cost thấp nhất. Vì nó dựa vào ước tính nên nếu statistics không chính xác hoặc dữ liệu thay đổi nhiều, planner có thể chọn plan chưa tối ưu. Tóm lại, query planner dùng cost-based optimizer để chọn cách chạy query hiệu quả nhất dựa trên ước tính chi phí.

[EN] In PostgreSQL, the query planner is the component that decides how to execute an SQL statement, for example whether to use an index scan or sequential scan, and which join method to apply. The cost-based optimizer estimates the “cost” of different execution plans based on data statistics, estimated row counts, I/O cost, and CPU cost, then chooses the plan with the lowest cost. Since it relies on estimation, if statistics are outdated or data changes significantly, the planner may choose a suboptimal plan. In short, the query planner uses a cost-based optimizer to select the most efficient way to run a query based on estimated costs.

[VN] Statistics (ANALYZE) ảnh hưởng thế nào đến execution plan?

[EN] How do statistics (ANALYZE) affect the execution plan?

[VN] Statistics trong PostgreSQL được thu thập bởi lệnh ANALYZE và dùng để ước tính số dòng, độ phân bố dữ liệu và độ chọn lọc của điều kiện WHERE. Những thông tin này ảnh hưởng trực tiếp đến execution plan vì cost-based optimizer dựa vào đó để tính toán chi phí và chọn index scan, sequential scan hay kiểu join phù hợp. Nếu statistics cũ hoặc không chính xác, database có thể ước tính sai số dòng và chọn plan kém hiệu quả. Tóm lại, statistics càng chính xác thì execution plan càng tối ưu.

[EN] In PostgreSQL, statistics are collected by the ANALYZE command and are used to estimate row counts, data distribution, and selectivity of WHERE conditions. This information directly affects the execution plan because the cost-based optimizer relies on it to calculate costs and choose between index scans, sequential scans, or join methods. If statistics are outdated or inaccurate, the database may misestimate row counts and select a poor plan. In short, more accurate statistics lead to better execution plans.

[VN] Cách đọc execution plan để tìm bottleneck?

[EN] How to read an execution plan to identify bottlenecks?

[VN] Để đọc execution plan và tìm bottleneck trong PostgreSQL, nên dùng EXPLAIN ANALYZE để xem cả kế hoạch ước tính và thời gian thực tế. So sánh estimated rows với actual rows để phát hiện chỗ ước tính sai; nếu chênh lệch lớn, statistics có thể không chính xác. Tập trung vào node có cost cao, actual time lớn hoặc số loop nhiều, vì đó thường là điểm nghẽn. Kiểm tra xem có sequential scan không cần thiết, sort tốn nhiều thời gian, hoặc join xử lý quá nhiều dòng. Tóm lại, tìm bước nào tốn nhiều thời gian hoặc xử lý nhiều dữ liệu nhất để xác định bottleneck.

[EN] To read an execution plan and find bottlenecks in PostgreSQL, use EXPLAIN ANALYZE to see both estimated plans and actual execution time. Compare estimated rows with actual rows to detect misestimation; large differences may indicate outdated statistics. Focus on nodes with high cost, high actual time, or many loops, as they are often

bottlenecks. Check for unnecessary sequential scans, expensive sorts, or joins processing too many rows. In short, identify the step that consumes the most time or processes the most data to find the bottleneck.

[VN] Làm sao phát hiện slow query trong production (pg_stat_statements, log_min_duration_statement)?

[EN] How to detect slow queries in production (pg_stat_statements, log_min_duration_statement)?

[VN] Để phát hiện slow query trong production, có thể dùng extension pg_stat_statements để thống kê các query theo tổng thời gian, số lần chạy và thời gian trung bình, từ đó tìm ra query tốn nhiều tài nguyên nhất. Ngoài ra, cấu hình log_min_duration_statement trong PostgreSQL để ghi log những query chạy lâu hơn một ngưỡng nhất định (ví dụ 500ms hoặc 1s). Bằng cách phân tích log và dữ liệu từ pg_stat_statements, bạn có thể xác định query chậm và ưu tiên tối ưu. Tóm lại, kết hợp thống kê tổng thể và logging theo thời gian thực để phát hiện và xử lý slow query hiệu quả.

[EN] To detect slow queries in production, you can use the pg_stat_statements extension to collect statistics about queries, such as total execution time, number of calls, and average time, helping you identify the most resource-consuming queries. You can also configure log_min_duration_statement in PostgreSQL to log queries that run longer than a specific threshold (for example, 500ms or 1s). By analyzing logs and pg_stat_statements data, you can find slow queries and prioritize optimization. In short, combine overall statistics and duration-based logging to effectively detect and handle slow queries.

[VN] Tại sao thiếu index có thể gây high CPU?

[EN] Why can missing indexes cause high CPU usage?

[VN] Thiếu index có thể gây high CPU vì database phải thực hiện sequential scan, tức là đọc toàn bộ bảng và kiểm tra từng dòng để tìm dữ liệu phù hợp. Khi bảng lớn hoặc query chạy nhiều lần, việc quét và so sánh từng dòng sẽ tiêu tốn rất nhiều CPU và bộ nhớ. Ngoài ra, nếu có nhiều join mà không có index ở cột liên kết, database phải xử lý nhiều phép so sánh hơn, làm tăng chi phí tính toán. Tóm lại, thiếu index khiến database phải làm nhiều công việc hơn trên toàn bộ dữ liệu, dẫn đến CPU usage cao.

[EN] Missing indexes can cause high CPU usage because the database must perform a sequential scan, meaning it reads the entire table and checks each row to find matching data. When the table is large or the query runs frequently, scanning and comparing every row consumes a lot of CPU and memory. In addition, joins without indexes on join columns require more comparisons, increasing computation cost. In short, without proper indexes, the database does much more work on all data, leading to high CPU usage.

[VN] Khi nào cần tăng work_mem hoặc shared_buffers?

[EN] When should work_mem or shared_buffers be increased?

[VN] Cần tăng work_mem khi query có nhiều thao tác sort, hash join hoặc aggregation lớn và bị tràn ra disk (thấy trong execution plan có “external sort” hoặc hash bị spill).

work_mem áp dụng cho từng operation, nên tăng quá cao có thể gây thiếu RAM khi nhiều query chạy cùng lúc. shared_buffers nên tăng khi database có nhiều dữ liệu được truy cập lặp lại và cần cache nhiều block trong bộ nhớ để giảm I/O disk. Tuy nhiên, tăng shared_buffers quá mức cũng có thể không hiệu quả nếu vượt quá khả năng RAM hệ thống. Tóm lại, tăng work_mem khi cần xử lý sort/hash lớn trong bộ nhớ, và tăng shared_buffers khi muốn cải thiện cache dữ liệu thường xuyên truy cập.

[EN] You should increase work_mem when queries perform large sorts, hash joins, or aggregations that spill to disk (for example, “external sort” or hash spill shown in the execution plan). work_mem applies per operation, so setting it too high can cause memory issues when many queries run at the same time. shared_buffers should be increased when the database frequently accesses the same data and needs more memory to cache blocks, reducing disk I/O. However, setting shared_buffers too high may not be effective if it exceeds available system RAM. In short, increase work_mem for large in-memory sort/hash operations, and increase shared_buffers to improve caching of frequently accessed data.

[VN] Khi nào cần connection pool như PgBouncer?

[EN] When is a connection pool like PgBouncer needed?

[VN] Cần connection pool như PgBouncer khi ứng dụng có nhiều request đồng thời và mở quá nhiều connection đến PostgreSQL, gây tốn tài nguyên và làm giảm hiệu năng. Mỗi connection trong PostgreSQL tiêu tốn bộ nhớ và CPU, nên nếu số lượng connection tăng cao (ví dụ từ web app hoặc microservices), database có thể bị quá tải dù query không nặng. PgBouncer giúp tái sử dụng connection, giảm chi phí tạo và đóng connection liên tục, và giới hạn số connection thực sự đến database. Tóm lại, dùng connection pool khi hệ thống có nhiều client hoặc traffic cao để bảo vệ và ổn định database.

[EN] You need a connection pool like PgBouncer when your application has many concurrent requests and opens too many connections to PostgreSQL, which consumes resources and reduces performance. Each PostgreSQL connection uses memory and CPU, so a high number of connections (for example from a web app or microservices) can overload the database even if queries are not heavy. PgBouncer reuses connections, reduces the cost of creating and closing them repeatedly, and limits the actual number of

connections to the database. In short, use a connection pool when your system has many clients or high traffic to keep the database stable and efficient.

[VN] Cách tối ưu connection cho workload cao?

[EN] How to optimize connections for high workloads?

[VN] Để tối ưu connection cho workload cao, nên dùng connection pool như PgBouncer để giới hạn và tái sử dụng connection thay vì mở quá nhiều connection trực tiếp đến database. Cấu hình số lượng connection tối đa phù hợp với CPU và RAM của server, tránh để quá cao gây quá tải. Ngoài ra, giữ transaction ngắn gọn để giải phóng connection nhanh, và tối ưu query để giảm thời gian giữ connection. Tóm lại, kiểm soát số lượng connection, dùng pool và đảm bảo mỗi connection được sử dụng hiệu quả là cách tối ưu cho workload cao.

[EN] To optimize connections for high workload, you should use a connection pool like PgBouncer to limit and reuse connections instead of opening too many direct connections to the database. Configure the maximum number of connections based on your server's CPU and RAM to avoid overload. Also keep transactions short to release connections quickly, and optimize queries to reduce connection holding time. In short, control the number of connections, use pooling, and ensure each connection is used efficiently for high workloads.

[VN] Autovacuum hoạt động ra sao và khi nào cần tuning?

[EN] How does autovacuum work and when does it need tuning?

[VN] Autovacuum trong PostgreSQL tự động dọn dẹp các bản ghi đã bị xóa hoặc cập nhật (dead tuples) và cập nhật statistics để tránh table bị phình to và giữ cho query planner hoạt động chính xác. Nó chạy nền dựa trên ngưỡng số lượng thay đổi trong bảng. Nếu autovacuum chạy quá chậm, bảng có nhiều dead tuples, query chậm hoặc xảy ra bloat, thì cần tuning bằng cách điều chỉnh các tham số như `autovacuum_vacuum_cost_limit`, `autovacuum_vacuum_scale_factor` hoặc tăng tài nguyên cho nó. Tóm lại, autovacuum giúp duy trì hiệu năng và cần tuning khi workload lớn hoặc dữ liệu thay đổi nhiều.

[EN] Autovacuum in PostgreSQL automatically cleans up dead tuples (rows that were deleted or updated) and updates statistics to prevent table bloat and keep the query planner accurate. It runs in the background based on thresholds of data changes in a table. If autovacuum is too slow, tables have many dead tuples, queries become slower, or bloat appears, you may need tuning by adjusting parameters like `autovacuum_vacuum_cost_limit`, `autovacuum_vacuum_scale_factor`, or allocating more resources. In short, autovacuum maintains performance and needs tuning when workload is high or data changes frequently.

[VN] Khi nào cần VACUUM và VACUUM FULL?

[EN] When is VACUUM and VACUUM FULL needed?

[VN] Trong PostgreSQL, VACUUM dùng để dọn dẹp các dead tuples sau khi DELETE hoặc UPDATE, giúp tái sử dụng không gian và duy trì hiệu năng; thường thì autovacuum sẽ tự chạy nên ít khi cần chạy tay, trừ khi bảng thay đổi nhiều và cần dọn gấp. VACUUM FULL không chỉ dọn dẹp mà còn thu hồi lại toàn bộ không gian đĩa bằng cách ghi lại bảng, giúp giảm kích thước file, nhưng nó khóa bảng trong suốt quá trình nên gây downtime tạm thời. Vì vậy, chỉ nên dùng VACUUM FULL khi bảng bị bloat lớn và cần lấy lại dung lượng đĩa ngay. Tóm lại, VACUUM để bảo trì thường xuyên, còn VACUUM FULL dùng khi cần thu hồi không gian lớn và chấp nhận lock bảng.

[EN] In PostgreSQL, VACUUM cleans up dead tuples after DELETE or UPDATE, allowing space reuse and maintaining performance; normally autovacuum handles this automatically, so manual VACUUM is only needed when there are heavy changes and urgent cleanup is required. VACUUM FULL not only cleans but also reclaims disk space by rewriting the whole table, reducing file size, but it locks the table during the process and can cause temporary downtime. Therefore, VACUUM FULL should only be used when the table has significant bloat and you need to reclaim disk space immediately. In short, use VACUUM for regular maintenance and VACUUM FULL when you must recover large disk space and can accept table locking.

[VN] CTE (WITH) ảnh hưởng thế nào đến performance ở các version PostgreSQL mới?

[EN] How do CTEs (WITH) affect performance in newer PostgreSQL versions?

[VN] Trong các version PostgreSQL cũ (trước 12), CTE (WITH) thường bị xem như “optimization fence”, nghĩa là luôn được materialize và thực thi độc lập trước, làm giảm khả năng tối ưu và có thể gây chậm. Tuy nhiên, từ PostgreSQL 12 trở đi, CTE không còn mặc định bị materialize nữa mà có thể được inline vào query chính nếu planner thấy có lợi, giúp tối ưu tốt hơn giống như subquery thông thường. Chỉ khi dùng từ khóa MATERIALIZED thì CTE mới bị ép materialize. Tóm lại, ở version mới, CTE linh hoạt và ít ảnh hưởng tiêu cực đến performance hơn, nhưng vẫn cần chú ý khi xử lý dữ liệu lớn.

[EN] In older PostgreSQL versions (before 12), CTE (WITH) was treated as an “optimization fence”, meaning it was always materialized and executed separately, which could reduce optimization and slow down queries. Starting from PostgreSQL 12, CTEs are no longer materialized by default and can be inlined into the main query if the planner decides it is beneficial, allowing better optimization similar to normal subqueries. Only when using the MATERIALIZED keyword will the CTE be forced to materialize. In short,

in newer versions, CTEs are more flexible and usually have less negative impact on performance, but care is still needed with large datasets.

[VN] Khi nào nên dùng materialized view?

[EN] When should materialized views be used?

[VN] Nên dùng materialized view khi có query phức tạp, tốn nhiều thời gian tính toán (ví dụ join nhiều bảng, aggregation lớn) và dữ liệu không cần realtime tuyệt đối. Materialized view lưu sẵn kết quả truy vấn ra đĩa, giúp đọc nhanh hơn so với tính toán lại mỗi lần. Tuy nhiên, nó cần được REFRESH định kỳ để cập nhật dữ liệu mới, và trong thời gian chưa refresh thì dữ liệu có thể cũ. Tóm lại, dùng materialized view khi cần tăng tốc đọc dữ liệu nặng và chấp nhận độ trễ cập nhật.

[EN] You should use a materialized view when you have complex and expensive queries (for example many joins or large aggregations) and the data does not need to be fully real-time. A materialized view stores the query result on disk, so reading it is much faster than recalculating each time. However, it must be refreshed periodically to update new data, and before refresh the data can be outdated. In short, use a materialized view to speed up heavy read queries when you can accept some data delay.

[VN] Khi nào nên denormalize để tăng hiệu năng?

[EN] When should denormalization be used to improve performance?

[VN] Nên denormalize khi hệ thống có workload đọc nhiều (read-heavy), query phức tạp với nhiều JOIN gây chậm, và hiệu năng quan trọng hơn tính chuẩn hóa tuyệt đối. Denormalize nghĩa là lưu trùng lặp một số dữ liệu để giảm số bảng cần JOIN, từ đó giảm I/O và CPU khi truy vấn. Tuy nhiên, cách này làm tăng rủi ro dữ liệu không đồng nhất và phức tạp hơn khi cập nhật (UPDATE/DELETE). Vì vậy, chỉ nên denormalize khi đã tối ưu index và query nhưng vẫn chưa đạt hiệu năng mong muốn. Tóm lại, denormalize để tăng tốc đọc dữ liệu khi chấp nhận đánh đổi về tính nhất quán và chi phí ghi.

[EN] You should denormalize when the system is read-heavy, queries are complex with many JOINS causing slow performance, and speed is more important than strict normalization. Denormalization means storing some duplicated data to reduce the number of JOINS, which lowers I/O and CPU usage during queries. However, it increases the risk of data inconsistency and makes updates (UPDATE/DELETE) more complex. Therefore, it should be used only after optimizing indexes and queries but still not meeting performance goals. In short, denormalize to improve read performance when you accept trade-offs in consistency and write complexity.

[VN] Khi nào cần full-text search với GIN index?

[EN] When is full-text search with a GIN index needed?

[VN] Nên dùng full-text search với GIN index khi cần tìm kiếm theo nội dung văn bản dài như bài viết, mô tả sản phẩm hoặc tài liệu, thay vì chỉ so sánh chính xác hoặc LIKE đơn giản. Full-text search cho phép tìm theo từ khóa, hỗ trợ stemming (biến thể của từ), và xếp hạng kết quả theo mức độ liên quan. GIN index giúp tăng tốc việc tìm kiếm trên tsvector bằng cách lập chỉ mục cho từng từ, rất hiệu quả khi dữ liệu lớn và truy vấn tìm kiếm thường xuyên. Tuy nhiên, GIN index tốn thêm dung lượng và chi phí ghi cao hơn khi INSERT hoặc UPDATE. Tóm lại, dùng full-text search với GIN khi cần tìm kiếm văn bản nhanh và thông minh trên dữ liệu lớn.

[EN] You should use full-text search with a GIN index when you need to search inside long text content such as articles, product descriptions, or documents, instead of simple exact match or basic LIKE queries. Full-text search allows keyword search, supports stemming (word variations), and ranks results by relevance. A GIN index speeds up searching on tsvector by indexing individual words, which is very efficient for large datasets and frequent search queries. However, GIN indexes use more storage and have higher write cost during INSERT or UPDATE. In short, use full-text search with GIN when you need fast and intelligent text searching on large data.

[VN] Làm sao tối ưu truy vấn có JOIN nhiều bảng lớn?

[EN] How to optimize queries with JOINS across multiple large tables?

[VN] Để tối ưu truy vấn JOIN nhiều bảng lớn, trước hết cần đảm bảo có index phù hợp trên các cột dùng để JOIN và điều kiện WHERE để giảm số dòng phải xử lý. Chỉ chọn những cột cần thiết thay vì SELECT *, và cố gắng lọc dữ liệu sớm (filter trước khi JOIN nếu có thể). Kiểm tra execution plan để xem kiểu join (nested loop, hash join, merge join) có phù hợp không và statistics có chính xác không. Ngoài ra có thể cân nhắc partition, materialized view hoặc denormalize nếu workload đọc rất lớn. Tóm lại, giảm số dòng tham gia JOIN và đảm bảo planner có đủ thông tin để chọn kế hoạch tốt nhất.

[EN] To optimize queries that JOIN many large tables, first ensure proper indexes exist on JOIN columns and WHERE conditions to reduce the number of processed rows. Select only necessary columns instead of using SELECT *, and try to filter data as early as possible (filter before JOIN when possible). Check the execution plan to see if the join method (nested loop, hash join, merge join) is appropriate and whether statistics are accurate. You may also consider partitioning, materialized views, or denormalization for heavy read workloads. In short, reduce the number of rows involved in JOINS and make sure the planner has enough information to choose the best plan.

[VN] Khi nào nên dùng partitioning để tăng hiệu năng truy vấn?

[EN] When should partitioning be used to improve query performance?

[VN] Nên dùng partitioning khi bảng rất lớn (hàng chục triệu đến hàng tỷ dòng) và query thường lọc theo một cột cụ thể như ngày tháng, range hoặc key rõ ràng. Partitioning giúp PostgreSQL chỉ quét các partition liên quan (partition pruning) thay vì toàn bộ bảng, từ đó giảm I/O và tăng tốc truy vấn. Nó cũng hữu ích khi cần quản lý dữ liệu theo từng phần, ví dụ xóa hoặc archive dữ liệu cũ nhanh hơn. Tuy nhiên, nếu bảng nhỏ hoặc query không lọc theo cột partition, thì partitioning có thể không mang lại lợi ích và còn tăng độ phức tạp. Tóm lại, dùng partitioning khi dữ liệu rất lớn và có pattern truy vấn rõ ràng theo một cột cụ thể.

[EN] You should use partitioning when a table is very large (tens of millions to billions of rows) and queries often filter by a specific column such as date, range, or a clear key. Partitioning allows PostgreSQL to scan only relevant partitions (partition pruning) instead of the whole table, reducing I/O and improving query speed. It is also useful for data management, such as quickly deleting or archiving old data. However, if the table is small or queries do not filter by the partition key, partitioning may not bring benefits and can increase complexity. In short, use partitioning when data is very large and queries follow a clear filtering pattern on a specific column.

[VN] Partition pruning hoạt động thế nào?

[EN] How does partition pruning work?

[VN] Partition pruning là cơ chế trong PostgreSQL giúp chỉ quét các partition liên quan thay vì toàn bộ bảng partitioned. Khi query có điều kiện WHERE trên cột partition key (ví dụ date hoặc id theo range/list), query planner sẽ phân tích điều kiện này và loại bỏ những partition không thể chứa dữ liệu phù hợp. Việc này có thể diễn ra ở thời điểm lập kế hoạch (planning time) hoặc khi thực thi (execution time), đặc biệt với prepared statement. Nhờ đó giảm I/O và số dòng phải xử lý, giúp truy vấn nhanh hơn. Tóm lại, partition pruning hoạt động bằng cách dùng điều kiện lọc để bỏ qua các partition không cần thiết.

[EN] Partition pruning is a mechanism in PostgreSQL that scans only relevant partitions instead of the entire partitioned table. When a query has a WHERE condition on the partition key (for example a date or id range/list), the query planner analyzes the condition and excludes partitions that cannot contain matching data. This can happen at planning time or execution time, especially with prepared statements. As a result, it reduces I/O and the number of rows processed, making queries faster. In short, partition pruning works by using filter conditions to skip unnecessary partitions.

[VN] Parallel query trong PostgreSQL hoạt động ra sao?

[EN] How does parallel query work in PostgreSQL?

[VN] Parallel query trong PostgreSQL cho phép một truy vấn được thực thi bởi nhiều worker process cùng lúc thay vì chỉ một process. Khi planner thấy bảng lớn và chi phí truy vấn cao, nó có thể tạo một leader process và nhiều worker để chia nhỏ công việc như parallel sequential scan, parallel hash join hoặc parallel aggregation. Các worker xử lý dữ liệu song song và gửi kết quả về cho leader để tổng hợp. Tính năng này giúp tận dụng nhiều CPU core và giảm thời gian thực thi, nhưng không phải mọi query đều dùng được (ví dụ query nhỏ hoặc có function không hỗ trợ song song). Tóm lại, parallel query chia công việc cho nhiều process để tăng tốc truy vấn lớn.

[EN] Parallel query in PostgreSQL allows a single query to be executed by multiple worker processes at the same time instead of just one process. When the planner detects a large table and high query cost, it can create a leader process and several workers to split tasks such as parallel sequential scan, parallel hash join, or parallel aggregation. The workers process data in parallel and send results back to the leader for final aggregation. This feature uses multiple CPU cores to reduce execution time, but not all queries can use it (for example small queries or functions that are not parallel-safe). In short, parallel query speeds up large queries by dividing the work among multiple processes.

[VN] Khi nào replication có thể ảnh hưởng read performance?

[EN] When can replication affect read performance?

[VN] Replication có thể ảnh hưởng read performance khi replica bị lag do ghi (write) quá nhiều trên primary, khiến replica phải áp dụng WAL liên tục và tiêu tốn CPU, I/O. Nếu replica vừa apply WAL vừa phục vụ nhiều query đọc nặng, tài nguyên có thể bị tranh chấp và làm chậm truy vấn. Ngoài ra, khi dùng synchronous replication, primary có thể phải chờ replica xác nhận, gián tiếp ảnh hưởng hiệu năng toàn hệ thống. Tóm lại, replication ảnh hưởng read performance khi tài nguyên replica không đủ hoặc khi có độ trễ và tải cao.

[EN] Replication can affect read performance when the replica has lag due to heavy writes on the primary, forcing it to continuously apply WAL and consume CPU and I/O. If the replica is both applying WAL and serving many heavy read queries, resource contention can slow down queries. In synchronous replication, the primary may wait for replica acknowledgment, indirectly impacting overall system performance. In short, replication affects read performance when replica resources are limited or when there is high load and replication lag.

[VN] Hard parse vs soft parse

[EN] Hard parse vs soft parse

[VN] Hard parse xảy ra khi database nhận một câu SQL mới mà chưa có execution plan trong cache, nên nó phải phân tích cú pháp (parse), kiểm tra quyền, và tạo execution plan

mới. Quá trình này tốn nhiều CPU và thời gian. Soft parse xảy ra khi câu SQL đã có execution plan trong cache, nên database chỉ cần tái sử dụng plan cũ mà không phải tạo lại. Vì vậy soft parse nhanh hơn và ít tốn tài nguyên hơn. Trong hệ thống hiệu năng cao, nên cố gắng giảm hard parse bằng cách dùng prepared statements hoặc query parameterization.

[EN] A hard parse happens when the database receives a new SQL statement that does not have an execution plan in the cache, so it must parse the query, check permissions, and generate a new execution plan. This process uses more CPU and takes more time. A soft parse happens when the SQL statement already has an execution plan in the cache, so the database can reuse the existing plan instead of creating a new one. Because of this, soft parse is faster and uses fewer resources. In high-performance systems, we try to reduce hard parses by using prepared statements or parameterized queries.

[VN] Khi nào dùng DynamoDB thay vì RDS?

[EN] When to use DynamoDB instead of RDS?

[VN] Nên dùng DynamoDB khi hệ thống cần scale rất lớn, độ trễ thấp và workload có nhiều read/write đơn giản theo key. DynamoDB phù hợp với mô hình key-value hoặc document, không cần join phức tạp và có thể auto scaling, high availability theo mặc định. Nó thích hợp cho hệ thống microservices, session store hoặc ứng dụng có traffic biến động mạnh. RDS phù hợp khi cần relational database với query phức tạp, join nhiều bảng, transaction ACID mạnh và ràng buộc dữ liệu chặt chẽ. Tóm lại, dùng DynamoDB khi ưu tiên scale và hiệu năng đơn giản; dùng RDS khi cần quan hệ và transaction phức tạp.

[EN] You should use DynamoDB when the system requires massive scalability, low latency, and simple read/write operations based on keys. DynamoDB fits key-value or document models, does not require complex joins, and provides built-in auto scaling and high availability. It is suitable for microservices, session storage, or applications with highly variable traffic. RDS is better when you need a relational database with complex queries, multi-table joins, strong ACID transactions, and strict data constraints. In summary, use DynamoDB when prioritizing scalability and simple performance; use RDS when relational logic and complex transactions are required.

[VN] DynamoDB vs OpenSearch

[EN] DynamoDB vs OpenSearch

[VN] DynamoDB là database NoSQL dạng key-value và document của AWS, được thiết kế cho truy cập dữ liệu rất nhanh và scale tự động. Nó phù hợp cho các hệ thống cần đọc/ghi nhanh bằng key như user profile, session, hoặc dữ liệu ứng dụng. Truy vấn trong DynamoDB thường dựa trên primary key hoặc index và không phù hợp cho tìm kiếm text phức tạp. OpenSearch (trước đây là Elasticsearch) là search engine và hệ thống phân tích

dữ liệu, được thiết kế để tìm kiếm full-text, log analysis, filtering và aggregation trên lượng dữ liệu lớn. Nó thường dùng cho search, monitoring, hoặc analytics. Tóm lại, DynamoDB dùng để lưu trữ và truy cập dữ liệu nhanh theo key, còn OpenSearch dùng để tìm kiếm và phân tích dữ liệu phức tạp.

[EN] DynamoDB is a NoSQL key-value and document database from AWS designed for very fast data access and automatic scaling. It is suitable for systems that need fast read/write by key, such as user profiles, sessions, or application data. Queries in DynamoDB usually rely on primary keys or indexes and are not designed for complex text search. OpenSearch (formerly Elasticsearch) is a search and analytics engine designed for full-text search, log analysis, filtering, and aggregation on large datasets. It is commonly used for search systems, monitoring, or analytics. In short, DynamoDB is used for fast key-based data storage and retrieval, while OpenSearch is used for powerful searching and data analysis.

III. Database Design & Optimization

Multi-tenant architecture nên thiết kế database như thế nào?

Làm sao viết test không phụ thuộc database thật?

Index hoạt động thế nào trong PostgreSQL?

Khi nào index làm chậm hệ thống?

Explain Analyze dùng để làm gì?

Isolation level gồm những gì?

Deadlock xảy ra khi nào?

Phân biệt optimistic vs pessimistic locking.

Transaction ACID là gì?

Partitioning trong PostgreSQL dùng khi nào?

So sánh Redis vs Memcached.

Redis Cluster và Redis Sentinel

Cache invalidation strategies?

N+1 query problem là gì?

Thiết kế schema thế nào để tránh N+1 query?

Khi nào dùng NoSQL như DynamoDB?

Cách thiết kế schema cho multi-tenant.

Replication trong PostgreSQL hoạt động thế nào?

Connection pool là gì? Tại sao quan trọng?

Seq Scan vs Index Scan khác nhau thế nào và khi nào Seq Scan lại nhanh hơn?

Index B-tree, Hash, GIN, GiST khác nhau thế nào và dùng khi nào?

Index có giúp cho LIKE hoặc ILIKE không?

Khi nào PostgreSQL không sử dụng index dù đã tạo?
Batch insert vs single insert khác nhau thế nào?
Làm sao tối ưu insert/update hàng loạt(bulk operation)?
Khi nào transaction quá dài gây vấn đề?
Lock và blocking ảnh hưởng performance ra sao?
Cách kiểm tra query đang bị lock?
Khi nào nên dùng composite index và thứ tự cột trong index quan trọng ra sao?
Covering index(index only scan) là gì?
Bitmap Index Scan dùng khi nào?
Subquery vs JOIN khác nhau về hiệu năng ra sao?
Làm sao tối ưu truy vấn có ORDER BY và LIMIT?
Làm sao tối ưu truy vấn có GROUP BY và aggregation lớn?
Window function ảnh hưởng hiệu năng thế nào?
Bạn hiểu thế nào về query planner và cost-based optimizer trong PostgreSQL?
Statistics(ANALYZE) ảnh hưởng thế nào đến execution plan?
Cách đọc execution plan để tìm bottleneck?
Làm sao phát hiện slow query trong production(pg_stat_statements, log_min_duration_statement)?
Tại sao thiếu index có thể gây high CPU?
Khi nào cần tăng work_mem hoặc shared_buffers?
Khi nào cần connection pool như PgBouncer?
Cách tối ưu connection cho workload cao?
Autovacuum hoạt động ra sao và khi nào cần tuning?
Khi nào cần VACUUM và VACUUM FULL?
CTE(WITH) ảnh hưởng thế nào đến performance ở các version PostgreSQL mới?
Khi nào nên dùng materialized view?
Khi nào nên denormalize để tăng hiệu năng?
Khi nào cần full-text search với GIN index?
Làm sao tối ưu truy vấn có JOIN nhiều bảng lớn?
Khi nào nên dùng partitioning để tăng hiệu năng truy vấn?
Partition pruning hoạt động thế nào?
Parallel query trong PostgreSQL hoạt động ra sao?
Khi nào replication có thể ảnh hưởng read performance?
Hard parse vs soft parse
Khi nào dùng DynamoDB thay vì RDS?
DynamoDB vs OpenSearch